AI Operations – Module 1 Assignment

Saketh Pranav (DA24B042)

Question 1 — Conceptual: Technical Debt Diagnosis

(1) (a) Entanglement (CACE) - Altering one feature or hyperparameter can silently shift the whole model's behaviour. Since changing the "estimated delivery time" feature's rounding logic unexpectedly hurt recommendations for a completely unrelated "favorite restaurants" feature it is Entanglement

(b) Undeclared Consumers - This kind of technical debt arises when other teams silently consume a model's output via a shared table, creating invisible dependencies that block safe changes. In this case, marketing dashboard team has read the model's raw output table for a campaign

(c) Configuration & Glue-Code Debt) - In this debt hyperparameters are scattered across scripts with no single source of truth; systems evolve into tangled "pipeline jungles."

(2) For (c), The solution for the mitigation here would be the replacement of the 14 shell scripts that are undocumented with a pipeline using DVC, which would have all the information on dependencies, outputs and commands. DVC supports the creation of pipelines where data is processed and then used to train models.

Question 2 — Applied: MLflow Experiment Comparison

For the sweep, I have chosen $hidden\_size = [32, 64, 128]$ and $learning\_rate = [0.01, 0.001]$

The best-performing run was mlp-h128-lr0.01, reaching 93.1% accuracy and 0.930 f1_macro — the highest of all six configurations. It paired the largest hidden layer (128 units) with the higher learning rate (0.01) and also produced the lowest final train_loss (0.053), suggesting that extra model capacity combined with faster learning converged to the best fit within the fixed 5-epoch budget.

Across all six runs, train_loss and val_accuracy move together rather than diverging: whichever run ends with the lowest train_loss also ends with the highest val_accuracy (e.g. h128-lr0.01: loss 0.053 $\rightarrow$ accuracy 0.931; h32-lr0.001: loss 0.343 $\rightarrow$ accuracy 0.906). No run shows a low train_loss paired with a stagnant or falling val_accuracy, so there's no sign of overfitting within 5 epochs — training and validation performance stayed aligned instead of splitting apart.

Learning rate had the larger effect on performance. Holding hidden_size fixed, switching from lr=0.001 to lr=0.01 improved accuracy by 1.1–2.2 points and cut train_loss by 0.19–0.20 in every case. Holding lr fixed, increasing hidden_size from 32 to 128 improved accuracy by only 1.3–1.4 points and train_loss by 0.09–0.10 — roughly half the impact. So while architecture (hidden_size) still helped, the learning rate was the dominant driver of both convergence speed and final accuracy across this sweep.